taoqick
博客等级 码龄13年
480 原创 1372 点赞 2734 收藏 659 粉丝
关注 私信
创作助手
大纲生成/代码生成/文章润色...
深度思考 (R1)

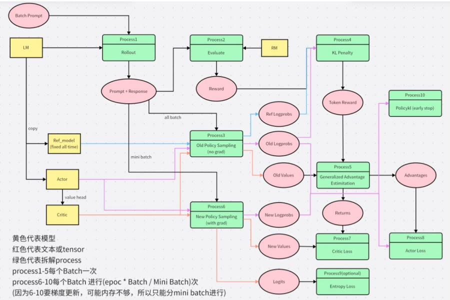
结合TRL的PPO的分析

转载 taoqick 已于 2023-10-11 21:20:38 修改 阅读量3k 收藏 15 点赞数 8
文章标签： 深度学习 机器学习 人工智能
2048 AI社区 文章已被社区收录 加入社区

摘要 本文深入探讨PPO算法，介绍其loss组成、训练关注指标及模型优化。阐述PPO算法全称、设计目的，对比DQN、TRPO等算法的优势，还分析了clip函数作用、数据效率保证方式，以及值函数与优势函数区别、探索策略实现和连续动作空间处理方法。

摘要生成于 C知道，由 DeepSeek-R1 满血版支持， 前往体验 >

仔细看了看TRL的code(https://github.com/huggingface/trl/blob/main/trl/trainer/ppo_trainer.py)，step大致流程为先batched_forward_pass，再过minibatch：



黄色代表模型
红色代表文本或tensor
绿色代表拆解process
process1-5每个Batch一次
process6-10每个Batch 进行(epoc * Batch / Mini Batch)次
(因为6-10要梯度更新，可能内存不够，所以只能分mini batch进行)

先写一写自己的理解

PPO的loss由以下几部分相加得到：

- actor的loss，代码里叫pg_loss，pg_loss是由-advantage值*exp(logprobs - old_logprobs)的得到的（刚好就是除的形式），至于advantage就是PPO里TD误差的加权和。这里值得注意的是PPO中的核心改进就是上面exp(logprobs - old_logprobs)控制在[1-\epi和1+\epi]之间做了CLIP。另外需要注意的是advantage是由rewards来得到的，rewards里增加了kl_penalty
- critic的loss，代码里叫vf_loss（value function loss），我们都有一个由Critic预测出的预期回报值V，以及一个真实的回报值G（returns），这俩MSE就是critic的loss。值得注意的是由大batch得到的values和小batch得到的vpreds也做了clip，values就是由critic模型输出的那个values。values和logits都是由LLM输出，代码中会给LLM加一个PreTrainedModelWrapper
- (optional) LM的loss，这个在TRL库里没有
- (optional) actor里可能会加一个entropy loss来让每一步动作更加均匀，但是在TRL的实现中并没有找到这个。

PPO训练中loss下降不能代表全部问题，以下几个指标也经常关注：

- reward系列： 希望reward可以平稳上升，reward->advantage->pg_loss
- kl系列：期望和ref模型距离不要太远
- PPL（perplexity系列）：语言模型

PPO过程中有4个模型，这4个都可以放一个大语言模型，导致显存要占4倍，也可以做一些优化：

- SFT Model: 一般会把LM做一遍SFT，保证Policy Model距离语言模型不要太远
- Policy Model: 最终得到的LM
- Reward Model(RM)：怎么评估rewards，例如计算器简单来说就是结果对不对，对得1分，不对得0分；对于toxic来说就是有没有毒
- Value Model：也可以叫Critic Model，评估当前状态下的期望收益。这个可以给Policy Model加一个Linear，省一些模型空间
下图来自论文Secrets of RLHF in Large Language Models Part I: PPO：

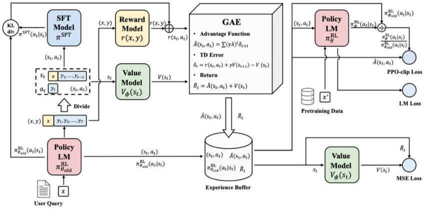


Figure 1: PPO workflow, depicting the sequential steps in the algorithm's execution. The process begins with sampling from the environment, followed by the application of GAE for improved advantage approximation. The diagram then illustrates the computation of various loss functions employed in PPO, signifying the iterative nature of the learning process and the policy updates derived from these losses.

一图拆解RLHF中的PPO

以下转载自：强化学习从零到RLHF（八）一图拆解RLHF中的PPO

众所周知，chatgpt的RLHF三阶段分别是：

1、SFT，也就是 Supervised Fine-Tuning，是 GPT-3 以及 ChatGPT 训练过程中的一个重要步骤。在这个步骤中，采用有监督的方式对预训练的语言模型进行微调。这又被称为行为克隆（Behavioral Cloning，简称BC），即直接使用专家的行为数据（例如，专家在特定情况下采取的动作）来训练模型。在行为克隆中，模型的目标是尽可能地复制专家的行为，而不是尝试优化某种奖励函数，所以它可能无法处理那些专家数据中没有覆盖到的情况，因为它完全依赖于专家的行为

基于 Graviton 的 Amazon EC2 T4g免费试用中，实现更高性价比
2核2G ARM架构云服务器 每月750小时免费试用 灵活弹性又可靠 早用早省～
广告

TA的精选
MetaCLIP、SigLip、EvaCLIP、XCLIP
679 阅读
Numerical Difference between vLLM logprobs and huggingface logprobs
117 阅读
对角矩阵的性质（diagonal matrix）
104105 阅读
Linux系统下安装TensorFlow的GPU版本
29719 阅读
Python创建dict的几种方法
27198 阅读
查看更多 >

大家在看
微服务远程调用完全透传实现：响应式与非响应式解决方案 1764
【YOLOv13多模态融合改进】| Arxiv 2024 DEYOLO：利用双增强机制和双向解耦聚焦模块，构建跨模态特征融合与单模态优化的完整框架 6
知识蒸馏 Knowledge Distillation 论文 Generalized Knowledge Distillation (GKD) 目标函数的演化 208
【机器学习&深度学习】多模态典型任务与应用全景
基于Python与Tkinter的校园点餐系统设计与实现

本
29719 阅读
Python创建dict的几种方法
27198 阅读
查看更多 >

大家在看
微服务远程调用完全透传实现：响应式与非响应式解决方案 1764
【YOLOv13多模态融合改进】| Arxiv 2024 DEYOLO：利用双增强机制和双向解耦聚焦模块，构建跨模态特征融合与单模态优化的完整框架 6
知识蒸馏 Knowledge Distillation 论文 Generalized Knowledge Distillation (GKD) 目标函数的演化 208
【机器学习&深度学习】多模态典型任务与应用全景
基于Python与Tkinter的校园点餐系统设计与实现

广告
CricGiri LIVE
FEEL THE THRILL!
Watch Now
Ads by MoMAGIC

相关专栏
深入理解 ChatGPT 专栏
113 人学习
ChatGPT专栏涵盖人工智能、机器学习...
一切皆是映射:AI人工智... 专栏
32 人学习
一切皆是映射:AI人工智能与大数据原理...
AGI通用人工智能 专栏
83 人学习
专注在 AGI通用人工智能 方面的科技

数据。

2、RM，奖励模型（Reward Model）是ChatGPT训练过程的第二阶段，它的目标是训练一个模型来适应人类的偏好。在这个阶段，首先从提示库中进行采样，并使用大型语言模型生成多个响应。然后，人工对这些响应进行排名，根据这些排名训练一个奖励模型。奖励模型的目标是学习人类对于不同响应的偏好，并将这些偏好编码到模型中。这样，奖励模型可以用来为模型生成的新响应打分，从而在后续的训练中引导模型生成更符合人类偏好的内容。这种方式不仅能帮助模型处理训练数据中未覆盖的情况，也能减少模型生成不确定或模棱两可的回答，从而打破行为克隆的影响。

3、PPO，PPO（Proximal Policy Optimization）是一种强化学习算法，它通过引入奖励信号来调整模型的行为，使模型生成的内容更符合人类的偏好。具体来说，PPO通过最大化预期奖励来调整模型的策略，使模型在选择行为时更倾向于选择可以得到更高奖励的行为。在这个阶段中，我们首先使用在第一阶段训练的有监督微调模型和第二阶段训练的奖励模型来生成一个初始的策略。然后，我们使用PPO算法来调整这个策略，使模型在生成内容时更考虑人类的偏好。通过这个阶段的训练，模型不仅可以理解人类的语言，还可以理解人类的偏好，并生成更符合人类偏好的内容。

本节我们将介绍目前在整个RLHF流程中，他的第三阶段强化学习是怎么做的。

PPO整体架构

宏观表述

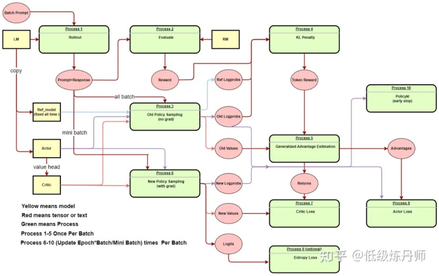
PPO 是一种用于训练强化学习模型的算法。它可以用于调整语言模型，使得模型生成的结果更符合人类的偏好。具体来说，过程可以分为三个阶段：

1. **Rollout and Evaluation**：在这个阶段，我们从prompt库里抽样，使用语言模型生成response，然后使用奖励模型（Reward Model, RM）给出奖励得分。这个得分反映了生成的response的质量，比如它是否符合人类的偏好，是否符合任务的要求等。
2. **Make experience**：在这个阶段，我们收集了一系列的“经验”，即模型的行为和对应的奖励。这些经验包括了模型生成的response以及对应的奖励得分。这些经验将被用于下一步的优化过程。
3. **Optimization**：在这个阶段，我们使用收集到的经验来更新模型的参数。具体来说，我们使用PPO算法来调整模型的参数，使得模型生成的response的奖励得分能够增加。PPO算法的一个关键特性是它尝试保持模型的行为不会发生太大的改变，这有助于保证模型的稳定性。

通过这三个阶段的微调，我们可以使得语言模型的输出更符合我们的期望，例如更有创造性，更符合人类的偏好等。

微观拆解

为了将transformer的ppo过程更为清晰的表达出来，我将整个过程拆解为了10个步骤，如下图所示：



知乎 @低级炼丹师

十个步骤依次是：

1. **Rollout**：根据策略（LM）生成轨迹（文本）。
2. **Evaluate**：对生成的轨迹进行评估（RM）。
3. **Old Policy Sampling**：从旧的策略（initial actor）中采样概率等信息。
4. **KL Penalty**：计算当前策略和原始LM之间的KL散度，用作对策略改变过快的惩罚项。
5. **Generalized Advantage Estimation (GAE)**：GAE是一种优势函数的估计方法，它结合了所有可能的n-step 进行advantage估计。
6. **New Policy Sampling**：从新的策略中采样概率等信息。
7. **Critic Loss**：Critic的目标是估计状态的价值函数，Critic loss就是价值函数预测值和实际回报之间的差距。
8. **Actor Loss**：Actor的目标是优化策略，Actor loss就是基于优势函数的策略梯度。
9. **Entropy Loss**：为了增加探索性，通常会添加一个基于策略熵的正则项，它鼓励策略保持多样性。
10. **Policykl**：这是对策略迭代过程的一个度量，它度量新策略和旧策略之间的差距。

在图中，我已经把所有的模型和变量都写出来了，其中黄色代表模型，红色代表值，绿色是步骤。朝向步骤的箭头表面该步骤需要该值或者模型，朝向步骤外部的箭头表示该步骤的输出。

下面我们开始依次讲解这十个步骤：

Rollout

介绍

在强化学习中，Rollout是指在给定的策略下模拟环境的过程。在PPO中，Rollout的过程对应于根据当前的语言模型（策略）生成文本（轨迹）。

这个过程依赖于在prompt库中抽取的一个batch的数据Batch Prompt和当前的语言模型LM。

语言模型接收一个prompt作为输入，并生成一个Response。这些Response就构成了我们的"轨迹"。

输入输出

输入： Batch Prompt、LM

输出： Prompt+Response

代码

在代码中，这个过程如下，我们使用huggingface的官方强化学习库TRL作为示例：

```
# 创建PPOTrainer实例。需要提供的参数包括：配置信息，模型，引用模型，分词器，数据集，数据整理
ppo_trainer = PPOTrainer(
    config,  # 配置信息
    model,  # 要训练的模型
    ref_model=None,  # 引用模型，通常是微调之前的预训练模型
    tokenizer=tokenizer,  # 用于文本编码和解码的分词器
    dataset=dataset,  # 用于训练的数据集
    data_collator=collator,  # 用于批量处理数据的数据整理器
    optimizer=optimizer,  # 用于模型优化的优化器
)

# 定义生成模型响应时的参数
generation_kwargs = {
    # "min_length": -1,  # 最小生成长度
    "top_k": 0.0,  # 在生成时，仅考虑前k个最可能的词
    "top_p": 1.0,  # 在生成时，仅考虑概率累计到某个阈值的词
    "do_sample": True,  # 是否进行抽样
    "pad_token_id": tokenizer.pad_token_id,  # 填充词的词ID
    "eos_token_id": 100_000,  # 句子结束词的词ID
}

# 使用定义好的参数生成模型的响应
response_tensors = ppo_trainer.generate(
        question_tensors,  # 输入的问题
        return_prompt=False,  # 是否返回提示
        length_sampler=output_length_sampler,  # 输出长度的抽样器
        **generation_kwargs,  # 生成参数
    )

# 将生成的响应从张量转换为文本，并存储在batch字典中
batch["response"] = tokenizer.batch_decode(response_tensors, skip_special_toker


# 这里将问题和相应的回答拼接起来，然后准备对拼接后的文本进行情感打分
texts = [q + r for q, r in zip(batch["query"], batch["response"])]
```

Evaluate

介绍

Evaluate是在强化学习中对生成的轨迹（在我们的例子中就是文本）进行评估的步骤。在PPO中，这个评估过程由一个RM模型来完成，来为每一对Prompt+Response产生一个标量奖励值，这个值表示生成的轨迹的好坏，优化过程会试图最大化这个值。

输入输出

输入：Prompt+Response、RM

输出：Reward

代码

```
# 创建一个情感分析pipeline，也就是RM模型。需要提供的参数包括：模型类型，模型名称，设备映射，
sentiment_pipe = pipeline(
    "sentiment-analysis",  # 模型类型，这里是情感分析
    model=reward_model_name,  # 模型名称
    device_map={"": current_device},  # 设备映射，这里将模型加载到当前设备
    model_kwargs={"load_in_8bit": True},  # 模型参数，这里是加载8位模型
    tokenizer=tokenizer,  # 用于文本编码和解码的分词器
)

# 使用情感分析对文本进行打分
pipe_outputs = sentiment_pipe(texts, **sent_kwargs)  # 'texts'是之前拼接好的问题和

# 计算奖励值。这里的奖励值是情感分析的得分减去一个基线值。
```

```
rewards = [torch.tensor(output[0]["score"] - script_args.reward_baseline) for
```

Old Policy Sampling

介绍

Old Policy Sampling 这个步骤是make experience的过程，计算并存储旧策略的概率、价值等值，来为后面更新的过程服务。

Old Logprobs

这个步骤中，我们从“旧的”策略，即在这个batch数据中初始的LM（initial actor）中计算每个token在旧的策略下的概率Old Logprobs。

```
actor：从策略网络拷贝来的模拟整个智能体在环境中行动的网络，
详细看本系列第五章中Actor-Critic经典架构小节
```

这个步骤的重要性在于，我们在优化策略的时候，需要比较新旧策略下动作的概率，以此来更新我们的策略。因此，我们需要存储旧的策略的动作概率作为参考。

```
之所以要比较这个概率是为了算一个叫ratio的值，用这个值更新策略梯度，能限制更新率，
详细看本系列第七章中PPO损失函数小节
```

Old Values

Old Values的含义是旧策略中每个时间步（每个token的预测结果）的价值，这个值由critic网络进行预测，critic网络就是actor上加几个线性层能够给每个token预测一个值。需要这个值的原因是advantage的计算依赖于Old Values。

```
critic：专门用来预测actor轨迹每一步价值的网络，
详细看本系列第五章中Actor-Critic经典架构小节
```

Ref Logprobs

Ref Logprobs的含义是最最原始的LM对于每个时间步的概率预测，一般就是固定不变的gpt3，计算这个值的目的是限制actor的更新，防止其偏离原始gpt3太远，他的实现在下一个步骤中。

输入输出

输入： Ref_model、Actor、Critic、Prompt+Response

输出： Ref Logprobs、Old Logprobs、Old Values

代码

```
all_logprobs, _, values, masks = self.batched_forward_pass(self.model, queries,
ref_logprobs, _, _, _ = self.batched_forward_pass(self.ref_model, queries, resp
```

batched_forward_pass方法的代码就是一个前向计算的过程。

KL Penalty

介绍

在PPO 实现中，KL Penalty是在模型优化过程中添加的一个惩罚项，用于保证经过强化学习后的模型（新策略actor）不会过于偏离原始预训练模型（ref model）。

具体来说，首先使用微调过程中的模型（新策略actor）和预训练模型（ref model）来计算序列中每个词的对数概率。然后，我们计算两个模型输出之间的 Kullback-Leibler (KL) 散度，这是一种衡量两个概率分布差异的方法。该KL散度被用作一个额外的奖励信号，并作为优化过程中的惩罚项，用于确保微调后的模型生成的响应不会偏离太远于预训练模型。这样可以保证模型在微调的过程中不会丢失预训练模型学习到的有用的知识和模式。

在图中的KL Penalty步骤中，我们会在reward上增加这个kl惩罚项来实现这个过程。

输入输出

输入： Ref Logprobs、Old Logprobs、Reward

输出： Token Reward

代码

```
# 初始化两个列表来分别存储奖励和非得分奖励
rewards, non_score_rewards = [], []

# 使用 zip 函数并行遍历输入的得分、对数概率、参考模型的对数概率以及mask
for score, logprob, ref_logprob, mask in zip(scores, logprobs, ref_logprobs, m
    # 计算 KL 散度，即模型的对数概率与参考模型的对数概率之间的差值
```

```
    kl = logprob - ref_logprob

    # 计算非得分奖励，即 KL 散度乘以 KL 控制器值的负值
    non_score_reward = -self.kl_ctl.value * kl
    non_score_rewards.append(non_score_reward)

    # 复制非得分奖励为新的奖励
    reward = non_score_reward.clone()

    # 找到mask中最后一个非零元素的索引，这表示输入序列的实际长度
    last_non_masked_index = mask.nonzero()[-1]

    # 对于最后一个非mask部分的token，其奖励是偏好模型的得分加上 KL 散度
    reward[last_non_masked_index] += score

    # 将计算的奖励添加到奖励列表中
    rewards.append(reward)

# 返回包含所有奖励的张量以及包含所有非得分奖励的张量
return torch.stack(rewards), torch.stack(non_score_rewards)
```

Generalized Advantage Estimation (GAE)

介绍

在强化学习中，我们会使用一个advantage的概念，来衡量每个时间步动作的含金量，他的本质含义是当前采样到的动作（生成的文本）的价值比平均的数学期望价值高的部分。

而在PPO中，我们一般使用GAE来进行advantage的计算。GAE是一种多步优势估计方法。它通过引入一个权衡参数λ，在单步TD误差和多步TD误差之间进行权衡，从而减小估计的方差，提高学习的稳定性。

GAE的主要目标是希望找到一种策略，使得从当前状态开始，采取该策略能够获得的未来奖励最大，GAE使用了一种名为TD误差的概念，这是一种预测未来奖励的方法。然后，GAE将这些TD误差组合成一个加权和，权重由一个衰减因子λ决定。当λ=0时，GAE就退化为普通的优势函数估计；当λ=1时，GAE就变成了一种名为"蒙特卡洛"的方法。总的来说，GAE的本质就是把优势估计为后续时间步TD误差的加权和，公式为：

$$A_t^{GAE} = \Sigma_{l=0}^{k-1}(\lambda\gamma)^l\delta_{t+l}$$

A_t^{GAE} 的含义是估计的是在t时刻的优势，k的含义是k步一更新，所以累加的TD误差加权和是k步之内的。λ 是权衡参数，用于在偏差和方差之间进行权衡。γ 是折扣因子，用于衰减未来的回报。δ_{t+l} 是在时间步t+l的TD误差，计算公式为 $r_{t+l+1} + \gamma V(s_{t+l+1}) - V(s_{t+l})$，这里的v是旧价值。在GAE步骤中，另外一个输出的值是returns，他的含义是每一步真实的后续累计折扣奖励，，这里使用advantage和value的和计算得到。

输入输出

输入： Token Reward、Old Values

输出： Advantages、Returns

代码

```
# 从后往前遍历整个生成的序列
for t in reversed(range(gen_len)):
    # 计算下一个状态的价值，如果当前状态已经是最后一个状态，则下一个状态的价值为0
    nextvalues = values[:, t + 1] if t < gen_len - 1 else 0.0

    # 计算 δ，它是奖励加上衰减后的下一个状态的价值，然后减去当前状态的价值
    delta = rewards[:, t] + self.config.gamma * nextvalues - values[:, t]

    # 使用 δ 更新 lastgaelam，这是 GAE 公式的一部分
    lastgaelam = delta + self.config.gamma * self.config.lam * lastgaelam

    # 将计算的优势值添加到优势值列表中
    advantages_reversed.append(lastgaelam)

# 将优势值列表反向并转换为张量
advantages = torch.stack(advantages_reversed[::-1]).transpose(0, 1)

# 计算回报值，它是优势值加上状态值
returns = advantages + values
```

❚❚ 停止滚动

New Policy Sampling

介绍

New Policy Sampling是PPO算法中的一个关键步骤。在PPO中，策略优化的过程涉及到两个策略：一个是"旧的"策略，这是我们在开始每次优化迭代时使用的策略，另一个是"新的"策略，这是我们在优化过程中不断更新的策略。

New Policy Sampling就是在新的策略（更新后的actor）下对轨迹（文本）计算概率的过程。这个信息会被用于计算"Actor Loss"，也就是策略梯度的损失。在我们的步骤中，Old Logprobs是一次

性一个batch的数据计算的，这是因为在一个batch中旧策略都是不变的；而New Logprobs是一个mini batch计算一次，这是因为新策略每个mini batch变一次。

此外这个步骤还会输出New Values和Logits分别用于critic loss和entropy loss的计算。

输入输出

输入： Ref_model、Actor、Critic

输出： New Logprobs、New Values、Logits

代码

```
logprobs, logits, vpreds, _ = self.batched_forward_pass(
                        self.model, batch["queries"], batch["responses"], mode
                    )
```

Critic Loss

介绍

在强化学习中，Critic是一个模型，其任务是估计状态的价值函数，也就是预测从当前状态开始，通过遵循某个策略，期望能得到的总回报。Critic的训练目标是最小化它的预测价值与实际回报之间的差距。这个差距被称为Critic Loss。

Critic Loss通常通过均方误差（Mean Squared Error, MSE）来计算。对于每一个状态，我们都有一个由Critic预测出的预期回报值V（s），以及一个真实的回报值G（returns）。Critic Loss就是这两个值之间差的平方。在一个批量的数据中，Critic Loss是所有状态的这个差的平方的平均值。公式如下：$CriticLoss = E[(V(s) - G)^2]$

其中E[.]表示期望值，$V(s)$ 是Critic对状态s（这个时间步的token）的价值预测New Values，G是真实的回报值Returns。

通过最小化Critic Loss，Critic的预测能力会逐渐提升。因为Critic的预测结果会被用来估计每个行动的优势（Advantage），这个优势值又会被用来计算策略的更新（Actor Loss）。

输入输出

输入： New Values、Returns

输出： 梯度更新

代码

```
# 将价值函数的预测值裁剪到一个范围内
vpredclipped = clip_by_value(
            vpreds, values - self.config.cliprange_value, values + self.config
        )

# 计算裁剪前和裁剪后的价值函数损失
vf_losses1 = (vpreds - returns) ** 2
vf_losses2 = (vpredclipped - returns) ** 2

# 最终的价值函数损失是裁剪前和裁剪后损失的最大值的平均值的一半
vf_loss = 0.5 * masked_mean(torch.max(vf_losses1, vf_losses2), mask)

# 计算裁剪操作实际发生的频率
vf_clipfrac = masked_mean(torch.gt(vf_losses2, vf_losses1).double(), mask)
```

代码的作用是将vpreds裁剪到一个范围内，这个范围是由values - self.config.cliprange_value和values + self.config.cliprange_value确定的，其中values是原始的价值函数预测值，self.config.cliprange_value是裁剪的范围。目的是为了避免value的变化太快。

Actor Loss

介绍

Actor Loss是基于策略梯度的损失函数，用于优化 Actor（策略）。

在深度强化学习中，我们通常有两个主要的组成部分：Actor 和 Critic。Actor 是策略，它决定文本会被怎么样生成。Critic 则是我们的价值函数估计器，它预测我们从当前状态开始，如果遵循当前的策略，能够得到的未来回报。

Actor Loss 是我们用来优化 Actor 的损失函数。它的计算通常基于优势函数，优势函数表示在给定的状态下采取某个行动比遵循当前策略的期望回报要好多少。

在 PPO 中，我们使用一种称为 Importance Sampling 的技术来计算 Actor Loss。我们比较了在旧策略和新策略下行动的概率（Old Logprobs，New Logprobs），然后将这个比值（也就是 Importance Sampling 的权重）与优势函数Advantages相乘，得到了对 Actor Loss 的一个估计。

PPO的Actor loss如下：

$L^{CLIP}(\Theta) = min(r_t(\Theta)\hat{A}_t, clip(r_t(\Theta), 1-\epsilon, 1+\epsilon)\hat{A}_t)$，

其中

$r_t(\Theta) = \frac{\pi_\Theta(a_t|s_t)}{\pi_{\Theta_{old}}(a_t|s_t)}$ 是新旧策略的比率。$\hat{A}_t$ 是优势函数，clip 是剪裁函数，它将$r_t(\Theta)$限制在 $[1-\epsilon, 1+\epsilon]$ 范围内，ϵ 是一个超参数，通常设置为 0.1 或 0.2。

这个损失函数的目标是最大化策略的期望回报，同时限制新旧策略之间的差异。当新旧策略的比率 $r_t(\Theta)$超出 $[1-\epsilon, 1+\epsilon]$ 范围时，剪裁函数会限制其影响，防止策略更新过大。

输入输出

输入： Old Logprobs，New Logprobs、Advantages

输出： 梯度更新

代码

```
# 计算新旧策略下概率的比值
ratio = torch.exp(logprobs - old_logprobs)

# 计算未截断的策略梯度损失
pg_losses = -advantages * ratio

# 计算截断的策略梯度损失
pg_losses2 = -advantages * torch.clamp(ratio, 1.0 - self.config.cliprange, 1.0

# 选择两者中较大的作为最终的策略梯度损失
pg_loss = masked_mean(torch.max(pg_losses, pg_losses2), mask)

# 计算因为截断导致策略梯度损失改变的比例
pg_clipfrac = masked_mean(torch.gt(pg_losses2, pg_losses).double(), mask)
```

Entropy Loss

介绍

在信息论与概率统计中，熵(entropy)是表示随机变量不确定性的度量。在强化学习中，策略的熵可以表示为：

$$\mathcal{H}\left(\pi\left(\cdot \mid s_t\right)\right) = -\sum_{a_t} \pi\left(a_t \mid s_t\right) \log\left(\pi\left(a_t \mid s_t\right)\right) = \mathbb{E}_{a_t \sim \pi}\left[-\log\left(\pi\left(a_t \mid s_t\right)\right)\right]$$

一个策略的熵越大，意味着这个策略选择各个动作的概率更加“平均”。在PPO中，为了提高算法的探索能力，我们一般在actor的loss中增加一项策略熵，并乘以一个系数entropy_coef，使得在优化actor_loss的同时，让策略的熵尽可能大。一般我们设置entropy_coef=0.01。

设置这个是因为如果策略总是倾向于选择某些特定的文本生成方式，那么它可能会错过一些其他的文本生成方式带来的更好的奖励。通过增加熵的项，可以使策略在选择词汇时保持一定的随机性，从而有更多的机会去探索那些可能带来更好奖励的文本轨迹。

在我们的Entropy Loss步骤中，只需要Logits就能计算这个损失。

输入输出

输入： Logits

输出： 梯度更新

代码

```
entropy = -torch.sum(logits* torch.log(logits + 1e-9), dim=-1).mean()
```

Policykl

介绍

在PPO中，KL散度被用作一种约束，以确保在优化过程中新策略不会偏离旧策略太远。这是为了防止过度优化，因为过度优化可能会导致策略性能的大幅下降。

我们希望在优化目标函数的同时，满足以下的KL散度约束：

$$\mathrm{KL}\left[\pi_{\theta_{\text{old}}}(\cdot|s_t), \pi_\theta(\cdot|s_t)\right] \leq \delta$$

其中，δ是一个预设的阈值，$\pi_{\theta_{old}}$ 和 π_θ 分别是旧策略和新策略。

在代码中，每个mini batch都会进行early stop的判定，如果计算出的KL散度大于δ，那么就会停止这一轮的优化，以保证新策略不会偏离旧策略太远。

注意：KL表示KL散度，$\pi_\theta(\cdot|s_t)$ 表示在状态 s_t 下，由参数θ确定的策略产生的动作的概率分布。

输入输出

输入： Old Logprobs，New Logprobs

输出： 是否early stop

代码

```
# 计算旧策略和新策略之间的KL散度
policykl = masked_mean(old_logprobs - logprobs, mask)
# old_logprobs 是旧策略下行为的概率的对数, logprobs 是新策略下的对数概率
# masked_mean 函数计算差异 (old_logprobs - logprobs) 的平均值, 但只考虑mask中对应元素为

# 检查计算出的KL散度 (policykl) 是否大于目标KL散度 (self.config.target_kl) 的1.5倍
if policykl > 1.5 * self.config.target_kl:
    self.optimizer.zero_grad()  # 如果实际的KL散度超过了目标的1.5倍, 那么策略改变过多,
    early_stop = True  # 并设置early_stop标志为True, 表示应提前停止优化, 以防止策略从旧
```

资源推荐

最后的最后我再推荐一些实用的强化学习代码库吧

1. Stable Baselines3：Stable Baselines3是一个提供各种深度强化学习算法实现的库，包括PPO，DQN，A2C等。所有的算法都采用了相同的API，使得在不同的算法之间进行切换变得非常方便。此外，它还提供了许多实用的功能，如模型保存/加载，向量化的环境，多种预处理选项等。
2. Spinning Up in Deep RL：这是OpenAI开发的一个库，其中包含了诸如PPO，A2C，SAC等算法的实现。该库的目标是提供一个清晰、简洁、易于理解的深度强化学习算法实现。
3. Ray's Rllib：Rllib是一个高效的分布式强化学习库，支持PPO，A3C，DQN等算法。它可以很容易地与Ray的其他组件（如Tune和Serve）集成，从而提供了一个全面的深度学习和强化学习的解决方案。

这三个是纯强化学习的代码，如果是和RLHF相关的话，推荐：

1. lvwerra/trl：这是一个基于Hugging Face的transformers库的强化学习库，可以用于训练Transformer语言模型。它使用ppo进行训练。
2. CarperAI/trlx：trlX是一个分布式训练框架，专门用于使用强化学习微调大型语言模型，可以使用提供的奖励函数或奖励标签的数据集进行训练。这个框架也支持微调Hugging Face模型。
3. hpcaitech/ColossalAI Chat： 同上
4. microsoft/DeepSpeedExamples DeepSpeed-Chat：同上

编辑于 2023-06-12 19:45 · IP 属地美国

强化学习 (Reinforcement Learning) 深度学习（Deep Learning） ChatGPT

▲ 赞同 105 ▼ 6 条评论 分享 ❚❚ 停止滚动 收藏 申请转载 ···

影响PPO算法性能的10个关键技巧

以下转载自：影响PPO算法性能的10个关键技巧（附PPO算法简洁Pytorch实现）

0.引言

PPO算法(Proximal Policy Optimization)[1]是目前深度强化学习(DRL)领域，最广泛应用的算法之一。然而在实际应用的过程中，PPO算法的性能却受到多种因素的影响。本文总结了影响PPO算法性能的10个关键技巧，并通过实验结果的对比，来探究各个trick对PPO算法性能的影响。同时，**我们将代码开源在了github上**，分别提供了**PPO算法的离散动作空间实现和连续动作空间实现**（见下面github链接中的4.PPO-discrete和5.PPO-continuous（包括了Gaussian分布和Beta分布））

https://github.com/Lizhi-sjtu/DRL-code-pytorch
github.com/Lizhi-sjtu/DRL-code-pytorch

1.PPO算法概述

PPO算法的核心是使用如下策略损失函数：

$$L^{CLIP}(\theta)=\hat{\mathbb{E}}_t\left[\min\left(r_t(\theta)\hat{A}_t,\operatorname{clip}(r_t(\theta),1-\epsilon,1+\epsilon)\hat{A}_t\right)\right] \tag{1}$$

其中，

$$r_t(\theta)=\frac{\pi_\theta\left(a_t\mid s_t\right)}{\pi_{\theta_{\text{old}}}\left(a_t\mid s_t\right)} \tag{2}$$

表示新旧策略之间的比率。ϵ 为一个超参数，用来确保当利用同一批数据进行多次策略更新时，“新旧策略之间的差距不要太大”。一般情况下，我们设置 $\epsilon=0.2$ 。

(1)式中，$\hat{A}_t$ 为优势函数的估计，根据PPO的原始论文建议，我们一般使用GAE（generalized advantage estimation）[2]来计算优势函数

$$\hat{A}_t^{GAE(\gamma,\lambda)}=\sum_{l=0}^{\infty}(\gamma\lambda)^l\delta_{t+l}^V \tag{3}$$

其中，δ_t^V 为：

$$\delta_t^V = r_t + \gamma V_\omega(s_{t+1}) - V_\omega(s_t) \tag{4}$$

以上便是PPO原始论文的核心内容。其实在PPO的原始论文中，除了利用GAE计算优势函数外，并没有提到其他的实现细节和技巧。但是在实际的各种代码实现，例如Open AI Baseline、Open AI Spinning Up中，却包括了许许多多的“trick”,实验表明，这些trick都会在一定程度上影响PPO算法的性能。我在参考了《PPO-Implementation matters in deep policy gradients A case study on PPO and TRPO》[3]这篇论文，以及下面这篇博客后

https://iclr-blog-track.github.io/2022/03/25/ppo-implementation-details/
iclr-blog-track.github.io/2022/03/25/ppo-implementatio...

通过自己的亲身实践，总结了**影响PPO算法性能最关键的10个trick**，如下表所示：

Trick 1	Advantage Normalization
Trick 2	State Normalization
Trick 3 & Trick 4	Reward Normalization & Reward Scaling
Trick 5	Policy Entropy
Trick 6	Learning Rate Decay
Trick 7	Gradient clip
Trick 8	Orthogonal Initialization
Trick 9	Adam Optimizer Epsilon Parameter
Trick 10	Tanh Activation Function

我们把集成了上述10个trick后的PPO算法命名为**PPO-max**，而不使用这10个trick的最基础的PPO算法命名为**PPO-min**。我们在gym中的四个连续动作空间环境下（BipedalWalker-v3、HalfCheetah-v2、Hopper-v2、Walker2d-v2）分别进行了实验，训练结果对比如图1所示。

> 注：
> 1.关于Trick 3 & Trick 4，由于只能对reward进行一种操作，我们默认选择使用Reward Scaling。
> 2.因为是连续动作空间，因此我们默认使用Gaussian分布来输出动作。
> 3.我们在每个环境中都使用了3个随机种子进行实验，并使用seaborn画图，采用“滑动平均”的方法来平滑训练曲线，后文的实验结果图同理。

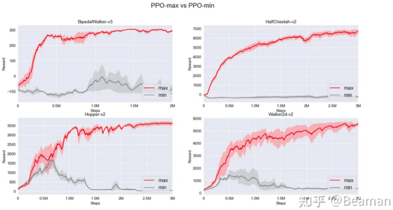
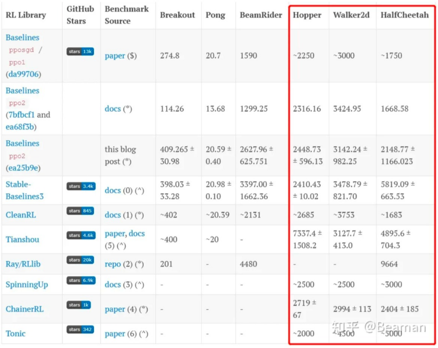


图1 PPO-max vs PPO-min

通过对比可以看出，不使用上述10个trick的**PPO-min**在四个gym环境上几乎无法训练，而**PPO-max**均可以到达非常理想的训练效果。在3M steps内，PPO-max在HalfCheetah-v2环境上reward可以达到**6700**，在Hopper-v2环境下可以达到**3600**，Walker2d-v2环境下可以达到**5500**。对比iclr-blog-track.github.io...这篇博客中给出的各个强化学习算法库的训练效果，除了“Tianshou”外，我们的实验结果均更胜一筹。



RL Library	GitHub Stars	Benchmark Source	Breakout	Pong	BeamRider	Hopper	Walker2d	HalfCheetah
Baselines `pposgd` / `ppo1` (da99706)	stars 13k	paper ($)	274.8	20.7	1590	~2250	~3000	~1750
Baselines `ppo2` (7bfbcf1 and ea68f3b)		docs (*)	114.26	13.68	1299.25	2316.16	3424.95	1668.58
Baselines `ppo2` (ea25b9e)		this blog post (*)	409.265 ± 30.98	20.59 ± 0.40	2627.96 ± 625.751	2448.73 ± 596.13	3142.24 ± 982.25	2148.77 ± 1166.023
Stable-Baselines3	stars 3.4k	docs (0) (^)	398.03 ± 33.28	20.98 ± 0.10	3397.00 ± 1662.36	2410.43 ± 10.02	3478.79 ± 821.70	5819.09 ± 663.53
CleanRL	stars 845	docs (1) (*)	~402	~20.39	~2131	~2685	~3753	~1683
Tianshou	stars 4.6k	paper, docs (5) (^)	~400	~20	-	7337.4 ± 1508.2	3127.7 ± 413.0	4895.6 ± 704.3
Ray/RLlib	stars 20k	repo (2) (*)	201	-	4480	-	-	9664
SpinningUp	stars 6.9k	docs (3) (^)	-	-	-	~2500	~2500	~3000
ChainerRL	stars 1k	paper (4) (*)	-	-	-	2719 ± 67	2994 ± 113	2404 ± 185
Tonic	stars 342	paper (6) (^)	-	-	-	~2000	~4500	~5000

图2 各种强化学习算法库的PPO训练结果

我们将代码开源在了github上，分别提供了PPO算法的离散动作空间实现和连续动作空间实现（见下面github链接中的4.PPO-discrete和5.PPO-continuous（包括了Gaussian分布和Beta分布），如果这份代码对您有帮助，欢迎您给一个star~）

https://github.com/Lizhi-sjtu/DRL-code-pytorch
github.com/Lizhi-sjtu/DRL-code-pytorch

2. 探究影响PPO算法性能的10个关键技巧

在这一节中，我们将逐一介绍上述PPO-max中10个trick的具体实现细节，并通过对比实验来探究这些trick究竟对PPO算法的性能有什么影响。（注：下面的实验讨论均已连续动作空间下Gaussian分布为例）

Trick 1—Advantage Normalization

在论文《The Mirage of Action-Dependent Baselines in Reinforcement Learning》[4]中提出了对advantage进行normalization的操作，可以提升PG算法的性能。具体代码实现层面，对advantage做normalization的方式主要有两种：

（1）batch adv norm：使用GAE计算完一个batch中的advantage后，计算整个batch中所有advantage的mean和std，然后减均值再除以标准差。

（2）minibatch adv norm：使用GAE计算完一个batch中的advantage后，不是直接对整个batch的advantage做normalization，而是在用每次利用minibatch更新策略之前，对当前这个minibatch中的advantage做normalization。（iclr-blog-track.github.io... 这篇博客中使用的就是minibatch adv norm）

是否使用advantage normalization，以及batch adv norm和minibatch adv norm的对比如图3所示。在我们的PPO-max中，默认使用的是batch adv norm（红色曲线）；如果关闭batch adv norm(棕色曲线)，PPO算法几乎无法训练，由此可见advantage normalization对PPO算法的性能有非常重要的影响。如果把batch adv norm替换成minibatch adv norm（黑色曲线），训练性能会有一定程度的下降。

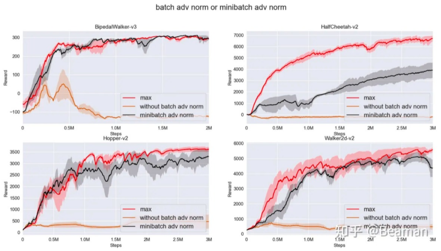


图3 batch adv norm VS minibatch dv norm

从原理角度分析，我个人认为如果是对每一个minibatch中的advantage单独做normalization，可能会导致每一次计算的mean和std有较大的波动，从而对算法的训练带来一定负面影响。因此，我们在这里建议使用**batch adv norm**的方式。

Trick 2—State Normalization

state normalization的核心在于，与环境交互的过程中，维护一个动态的关于所有经历过的所有state的mean和std, 然后对当前的获得的state做normalization。经过normalization后的state符合mean=0,std=1的正态分布，用这样的状态作为神经网络的输入，更有利于神经网络的训练。

具体实现方式如下：

（1）首先定义一个动态计算mean和std的class，名为**RunningMeanStd**。这个class中的方法的核心思想是已知n个数据的mean和std，如何计算n+1个数据的mean和std

> 注：使用这种方法计算mean和std的主要原因是，我们不可能事先初始化一个无限大的buffer，每获得一个新的状态数据后都重新计算buffer中所有数据的mean和std，这样太浪费计算资源和时间，也不现实。

```python
class RunningMeanStd:
    # Dynamically calculate mean and std
    def __init__(self, shape):  # shape:the dimension of input data
        self.n = 0
        self.mean = np.zeros(shape)
        self.S = np.zeros(shape)
        self.std = np.sqrt(self.S)

    def update(self, x):
        x = np.array(x)
        self.n += 1
        if self.n == 1:
```

```
            self.mean = x
            self.std = x
        else:
            old_mean = self.mean.copy()
            self.mean = old_mean + (x - old_mean) / self.n
            self.S = self.S + (x - old_mean) * (x - self.mean)
            self.std = np.sqrt(self.S / self.n )
```

（2）定义一个名叫**Normalization**的类，其中实例化上面的**RunningMeanStd**，需要传入的参数shape代表当前环境的状态空间的维度。训练过程中，每次得到一个state，都要把这个state传到Normalization这个类中，然后更新mean和std，再返回normalization后的state。

```
class Normalization:
    def __init__(self, shape):
        self.running_ms = RunningMeanStd(shape=shape)

    def __call__(self, x, update=True):
        # Whether to update the mean and std,during the evaluating,update=Flase
        if update:
            self.running_ms.update(x)
        x = (x - self.running_ms.mean) / (self.running_ms.std + 1e-8)

        return x
```

是否使用state normalization的对比如图4所示，红色曲线为PPO-max，蓝色曲线为在PPO-max的基础上关闭state normalization。通过对比可以看出，state normalization这个trick对PPO算法的整体性能有一定提升。

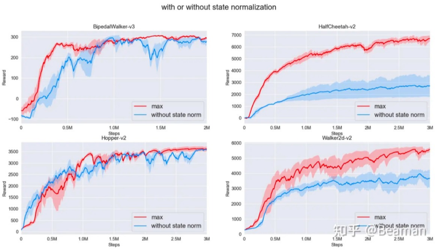


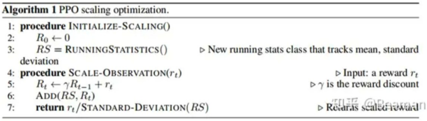
图4 是否使用state normalization

Trick 3 & Trick 4—— Reward Normalization & Reward Scaling

对reward的处理，目前有reward normalization和reward scaling两种方式：这两种处理方式的目的都是希望调整reward的尺度，避免因过大或过小的reward对价值函数的训练产生负面影响。

（1）**reward normalization**：与state normalization的操作类似，也是动态维护所有获得过的reward的mean和std，然后再对当前的reward做normalization。

（2）**reward scaling**：在《PPO-Implementation matters in deep policy gradients A case study on PPO and TRPO》[3]这篇论文中，作者中提出了一种名叫reward scaling的方法，如图5所示。reward scaling与reward normalization的区别在于，reward scaling是动态计算一个standard deviation of a rolling discounted sum of the rewards，然后只对当前的reward除以这个std。



Algorithm 1 PPO scaling optimization.

```
procedure INITIALIZE-SCALING()
    $R_0 \leftarrow 0$
    $RS$ = RUNNINGSTATISTICS()        ▷ New running stats class that tracks mean, standard deviation
procedure SCALE-OBSERVATION($r_t$)        ▷ Input: a reward $r_t$
    $R_t \leftarrow \gamma R_{t-1} + r_t$        ▷ $\gamma$ is the reward discount
    ADD($RS$, $R_t$)
    return $r_t$/STANDARD-DEVIATION($RS$)        ▷ Returns scaled reward
```

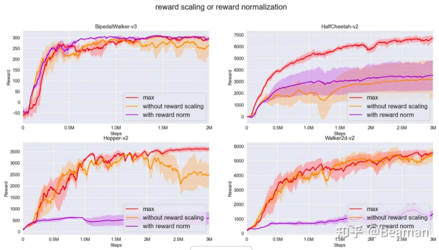
图5 PPO reward scaling

reward scaling的代码实现被集成在了**RewardScaling**这个class中，具体代码如下：

```
class RewardScaling:
    def __init__(self, shape, gamma):
        self.shape = shape  # reward shape=1
        self.gamma = gamma  # discount factor
        self.running_ms = RunningMeanStd(shape=self.shape)
        self.R = np.zeros(self.shape)

    def __call__(self, x):
        self.R = self.gamma * self.R + x
        self.running_ms.update(self.R)
        x = x / (self.running_ms.std + 1e-8)  # Only divided std
        return x
```

```
    def reset(self):  # When an episode is done,we should reset 'self.R'
        self.R = np.zeros(self.shape)
```

reward norm和reward scaling的对比如图6所示。图中，PPO-max(红色)中默认使用的是reward scaling，去掉reward scaling后（橙色），性能有一定程度下降；如果把PPO-max中的reward scaling 换成reward norm(紫色)，在HalfCheetah-v2、Hopper-v2和Walker-v2这三个环境下，对训练性能的伤害非常严重。**因此，我们建议使用reward scaling而不是reward normalization。**



自动滚动

图6 reward sca normalization

Trick 5—Policy Entropy

在信息论与概率统计中，熵(entropy)是表示随机变量不确定性的度量。在强化学习中，策略的熵可以表示为：

$$\mathcal{H}\left(\pi\left(\cdot \mid s_t\right)\right)=-\sum_{a_t} \pi\left(a_t \mid s_t\right) \log \left(\pi\left(a_t \mid s_t\right)\right)=\mathbb{E}_{a_t \sim \pi}\left[-\log \left(\pi\left(a_t \mid s_t\right)\right)\right] \quad (5)$$

一个策略的熵越大，意味着这个策略选择各个动作的概率更加“平均”。在PPO中，为了提高算法的探索能力，我们一般在actor的loss中增加一项策略熵，并乘以一个系数entropy_coef，使得在优化actor_loss的同时，让策略的熵尽可能大。一般我们设置entropy_coef=0.01。

是否使用策略熵的对比如图7所示，通过对比可以看出，使用策略熵可以一定程度提高训练效果，如果在PP~~O-max（红色）的基础上不适用策略熵（即令entropy_coef=0，图中粉色），~~reward往往会收敛到次优解。

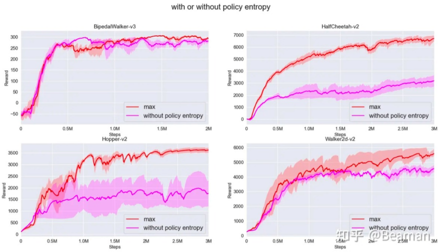


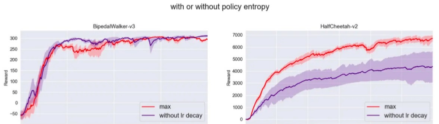
图7 是否使用策略熵

Trick 6—Learning Rate Decay

学习率衰减可以一定程度增强训练后期的平稳性，提高训练效果。这里我们采用**线性衰减**学习率的方式，使lr从初始的3e-4，随着训练步数线性下降到0，具体代码如下：

```
def lr_decay(self, total_steps):
    lr_a_now = self.lr_a * (1 - total_steps / self.max_train_steps)
    lr_c_now = self.lr_c * (1 - total_steps / self.max_train_steps)
    for p in self.optimizer_actor.param_groups:
        p['lr'] = lr_a_now
    for p in self.optimizer_critic.param_groups:
        p['lr'] = lr_c_now
```

是否使用学习率衰减的对比如图8所示，通过对比可以看出，学习率衰减这一trick的确对训练效果有一定帮助。



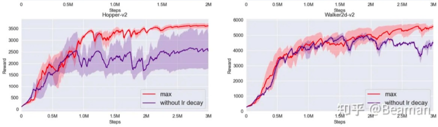


图8 是否使用学习率衰减

Trick 7—Gradient clip

梯度剪裁是为了防止训练过程中梯度爆炸从而引入的一项trick，同样可以起到稳定训练过程的作用。在Pytorch中，梯度剪裁只需要在更新actor和critic时增加一条语句即可实现。

```
# Update actor
self.optimizer_actor.zero_grad()
actor_loss.mean().backward()
if self.use_grad_clip: # Trick 7: Gradient clip
    torch.nn.utils.clip_grad_norm_(self.actor.parameters(), 0.5)
self.optimizer_actor.step()

# Update critic
self.optimizer_critic.zero_grad()
critic_loss.backward()
if self.use_grad_clip: # Trick 7: Gradient clip
    torch.nn.utils.clip_grad_norm_(self.critic.parameters(), 0.5)
self.optimizer_critic.step()
```

是否增加gradient clip的对比如图9所示，通过对比可以看出，gradient clip对训练效果有一定提升。

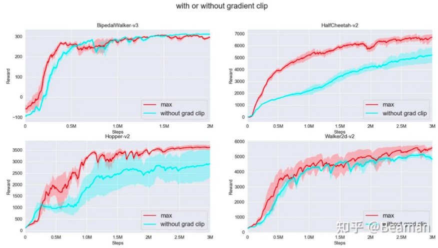


图9 是否增加gradient clip

Trick 8—Orthogonal Initialization

正交初始化（Orthogonal Initialization）是为了防止在训练开始时出现梯度消失、梯度爆炸等问题所提出的一种神经网络初始化方式。具体的方法分为两步：

（1）用均值为0，标准差为1的高斯分布初始化权重矩阵，

（2）对这个权重矩阵进行奇异值分解，得到两个正交矩阵，取其中之一作为该层神经网络的权重矩阵。

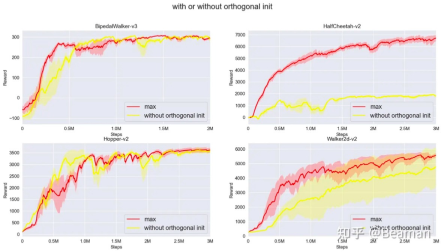
使用正交初始化的Actor和Critic实现如下面的代码所示：

> 注：
> 1. 我们一般在初始化actor网络的输出层时，会把gain设置成0.01，actor网络的其他层和critic网络都使用Pytorch中正交初始化默认的gain=1.0。
> 2. 在我们的实现中，actor网络的输出层只输出mean，同时采用nn.Parameter的方式来训练一个“状态独立”的log_std，这往往比直接让神经网络同时输出mean和std效果好。（之所以训练log_std，是为了保证std=exp(log_std)>0）

```
# orthogonal init
def orthogonal_init(layer, gain=1.0):
    nn.init.orthogonal_(layer.weight, gain=gain)
    nn.init.constant_(layer.bias, 0)

class Actor_Gaussian(nn.Module):
    def __init__(self, args):
        super(Actor_Gaussian, self).__init__()
        self.max_action = args.max_action
        self.fc1 = nn.Linear(args.state_dim, args.hidden_width)
        self.fc2 = nn.Linear(args.hidden_width, args.hidden_width)
        self.mean_layer = nn.Linear(args.hidden_width, args.action_dim)
        self.log_std = nn.Parameter(torch.zeros(1, args.action_dim))  # We use
        if args.use_orthogonal_init:
            print("------use_orthogonal_init------")
```

```
            orthogonal_init(self.fc1)
            orthogonal_init(self.fc2)
            orthogonal_init(self.mean_layer, gain=0.01)

    def forward(self, s):
        s = torch.tanh(self.fc1(s))
        s = torch.tanh(self.fc2(s))
        mean = self.max_action * torch.tanh(self.mean_layer(s))  # [-1,1]->[-ma
        return mean

    def get_dist(self, s):
        mean = self.forward(s)
        log_std = self.log_std.expand_as(mean)  # To make 'log_std' have the sa
        std = torch.exp(log_std)  # The reason we train the 'log_std' is to ens
        dist = Normal(mean, std)  # Get the Gaussian distribution
        return dist

class Critic(nn.Module):
    def __init__(self, args):
        super(Critic, self).__init__()
        self.fc1 = nn.Linear(args.state_dim, args.hidden_width)
        self.fc2 = nn.Linear(args.hidden_width, args.hidden_width)
        self.fc3 = nn.Linear(args.hidden_width, 1)
        if args.use_orthogonal_init:
            print("------use_orthogonal_init------")
            orthogonal_init(self.fc1)
            orthogonal_init(self.fc2)
            orthogonal_init(self.fc3)

    def forward(self, s):
        s = torch.tanh(self.fc1(s))
        s = torch.tanh(self.fc2(s))
        v_s = self.fc3(s)
        return v_s
```

是否使用正交初始化的对比如图所10示，通过对比可以看出，正交初始化对训练性能有一定提高。

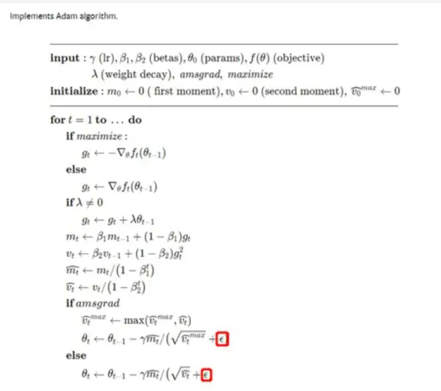


知乎 @Beaman

图10 是否使用正交初始化

Trick 9—Adam Optimizer Epsilon Parameter

pytorch中Adam优化器默认的eps=1e-8，它的作用是提高数值稳定性（pytorch官方文档中对Adam优化器的介绍如图11，eps即红框中的 ϵ ）



Implements Adam algorithm.

```
input : $\gamma$ (lr), $\beta_1, \beta_2$ (betas), $\theta_0$ (params), $f(\theta)$ (objective)
        $\lambda$ (weight decay), $amsgrad$, $maximize$
initialize : $m_0 \leftarrow 0$ ( first moment), $v_0 \leftarrow 0$ (second moment), $\widehat{v_0}^{max} \leftarrow 0$

for $t = 1$ to ... do
    if $maximize$ :
        $g_t \leftarrow -\nabla_\theta f_t(\theta_{t-1})$
    else
        $g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$
    if $\lambda \neq 0$
        $g_t \leftarrow g_t + \lambda\theta_{t-1}$
    $m_t \leftarrow \beta_1 m_{t-1} + (1-\beta_1) g_t$
    $v_t \leftarrow \beta_2 v_{t-1} + (1-\beta_2) g_t^2$
    $\widehat{m_t} \leftarrow m_t / (1-\beta_1^t)$
    $\widehat{v_t} \leftarrow v_t / (1-\beta_2^t)$
    if $amsgrad$
        $\widehat{v_t}^{max} \leftarrow \max(\widehat{v_t}^{max}, \widehat{v_t})$
        $\theta_t \leftarrow \theta_{t-1} - \gamma \widehat{m_t} / (\sqrt{\widehat{v_t}^{max}} + \boxed{\epsilon}$
    else
        $\theta_t \leftarrow \theta_{t-1} - \gamma \widehat{m_t} / (\sqrt{\widehat{v_t}} + \boxed{\epsilon}$
```

return θ_t

图11 Pytorch官方文档中Adam优化器介绍

在Open AI的baseline，和MAPPO的论文里，都单独设置eps=1e-5，这个特殊的设置可以在一定程度上提升算法的训练性能。

是否设置Adam优化器中eps参数的对比如图10所示，通过对比可以看出，单独设置Adam优化器的eps=1e-5可以一定程度提升算法性能。

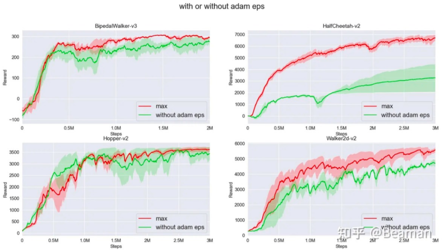


图12 是否设置Adam eps

Trick10—Tanh Activation Function

一般的强化学习算法，例如DDPG TD3 SAC等算法，都默认使用relu激活函数，但是经过实验表明，PPO算法更适合使用Tanh激活函数。在论文《PPO-Implementation matters in deep policy gradients A case study on PPO and TRPO》[3]中，作者也建议使用tanh激活函数。

使用tanh和relu激活函数的对比如图13所示，在PPO-max中我们默认使用的是tanh激活函数，把PPO-max中的激活函数替换成relu后（actor网络的最后一层mean_layer依然使用tanh），算法性能有了一定程度的下降。**因此我们建议PPO算法默认使用tanh激活函数。**

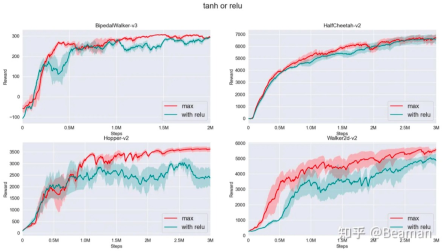


图13 激活函数tanh or relu

3.关于PPO算法的一些补充

3.1关于GAE的计算，以及done信号的区分

在gym的环境中，done=True有三种情况：

1. 游戏胜利(win)

2. 游戏失败（dead）

3. 达到最大步长

对于前两种情况，即dead or win的时候，我们可以认为当前的状态s就是终止状态，没有下一个状态s'的（例如BipedalWalker-v3这个环境，就会出现智能体通关或者摔倒的情形）。而对于第三种情况，即达到最大步长，这时本质上是人为"截断"了当前的回合，事实上当前的状态s并不是终止状态。因此我们在主循环中，需要对env.step()返回的done进行区分。为了区分前两种情况和第三种情况，我在程序中做了如下处理：

```
if done and episode_steps != args.max_episode_steps:
    dw = True
else:
    dw = False
```

我用dw这个变量来表示dead or win这两种情况，然后同时向Buffer中存储dw和done。在计算GAE时，对dw和done进行区分：

> if dw=True: deltas=r-v(s)
> if dw=False:deltas=r+gamma*v(s')-v(s)
> 当done=True时，说明出现了上述三种情况的其中之一，即意味着一个episode的结束。在逆序计算adv时，遇到done=True，就要重新计算。

使用GAE计算advangate的具体代码如下

```
s, a, a_logprob, r, s_, dw, done = replay_buffer.numpy_to_tensor()  # Get trai
"""
    Calculate the advantage using GAE
    'dw=True' means dead or win, there is no next state s'
    'done=True' represents the terminal of an episode(dead or win or reaching t
"""
adv = []
gae = 0
with torch.no_grad():  # adv and v_target have no gradient
    vs = self.critic(s)
    vs_ = self.critic(s_)
    deltas = r + self.gamma * (1.0 - dw) * vs_ - vs
    for delta, d in zip(reversed(deltas.flatten().numpy()), reversed(done.flat
        gae = delta + self.gamma * self.lamda * gae * (1.0 - d)
        adv.insert(0, gae)
    adv = torch.tensor(adv, dtype=torch.float).view(-1, 1)
    v_target = adv + vs
    if self.use_adv_norm:  # Trick 1:advantage normalization
        adv = ((adv - adv.mean()) / (adv.std() + 1e-5))
```

3.2 Guassian分布与Beta分布

一般的连续动作空间版本的PPO算法，都默认使用Gaussian分布来输出动作。在《Improving Stochastic Policy Gradients in Continuous Control with Deep Reinforcement Learning using the Beta Distribution 》[5]指出，由于Gaussian分布是一个无界的分布，我们在采样动作后往往需要clip操作来把动作限制在有效动作范围内，这个clip的操作往往会给算法性能带来负面影响。因此这篇论文提出采用一个有界的Beta分布来代替Guassain分布，通过Beta分布采样的动作一定在[0,1]区间内（Beta分布的概率密度函数曲线如图14所示），因此我们可以把采样到的[0,1]区间内的动作映射到任何我们想要的动作区间。

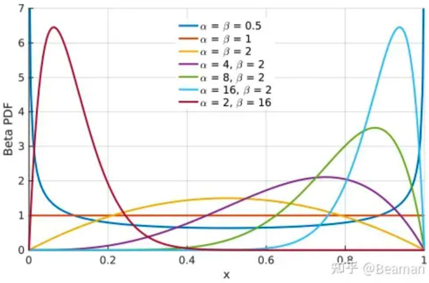


图14 Beta分布的概率密度函数曲线

在PPO中采用Beta分布的主要改动在于actor网络的定义，代码如下：

```
class Actor_Beta(nn.Module):
    def __init__(self, args):
        super(Actor_Beta, self).__init__()
        self.fc1 = nn.Linear(args.state_dim, args.hidden_width)
        self.fc2 = nn.Linear(args.hidden_width, args.hidden_width)
        self.alpha_layer = nn.Linear(args.hidden_width, args.action_dim)
        self.beta_layer = nn.Linear(args.hidden_width, args.action_dim)
        if args.use_orthogonal_init:
            print("------use_orthogonal_init------")
            orthogonal_init(self.fc1, gain=1.0)
            orthogonal_init(self.fc2, gain=1.0)
            orthogonal_init(self.alpha_layer, gain=0.01)
            orthogonal_init(self.beta_layer, gain=0.01)

    def forward(self, s):
        s = torch.tanh(self.fc1(s))
        s = torch.tanh(self.fc2(s))
        # alpha and beta need to be larger than 1,so we use 'softplus' as the a
        alpha = F.softplus(self.alpha_layer(s)) + 1.0
        beta = F.softplus(self.beta_layer(s)) + 1.0
        return alpha, beta

    def get_dist(self, s):
        alpha, beta = self.forward(s)
        dist = Beta(alpha, beta)
```

```
        return dist

    def mean(self, s):
        alpha, beta = self.forward(s)
        mean = alpha / (alpha + beta)  # The mean of the beta distribution
        return mean
```

在一些环境下，例如HalfCheetah-v2，采用Beta有时会比Gaussian获得更好的效果，如图15所示。

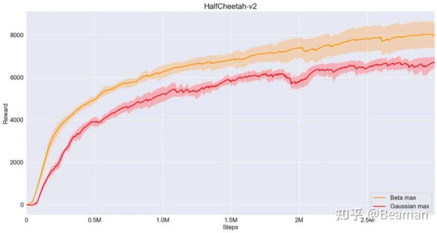


图15 Gaussian VS Beta

因此，我们在PPO算法的实现中，集成了Beta分布，可以通过设置'policy_dist'参数来实现Gaussian分布和Beta分布之间的切换。

4. 总结

在这篇文章中，我根据自己个人的实际经验，列出了影响PPO算法性能的10个关键技巧，并通过对比实验来探究这些技巧对PPO算法性能的具体影响，同时给出了完整的PPO算法的pytorch实现（包括了离散动作版本和连续动作版本（集成Gaussian分布和Beta分布））。

同时，我想额外指出的是，我上面列出的这10条trick，也不一定在所有的情况下都有效。很多时候，深度强化学习算法的训练是一件很“玄”的事情，不同的环境，不同的状态空间、动作空间和奖励函数的设计，都会对算法的性能产生不同的影响。同样的一个trick，可能在一个任务中效果很好，在另一个任务中却完全不work。我总结这10条trick，更多的是希望给正在学习和使用PPO算法的朋友提供一个整体方向上的引导，当你在用PPO算法解决一个实际任务但是效果不理想时，或许可以尝试一下我列出的这几条trick。当然，过渡迷恋于调参往往是不可取的，我们应该把更多的注意力放在模型算法本身的改进上。

最后，由于本人水平有限，上述文章内容和代码中难免存在错误，欢迎大家与我交流，批评指正！

参考

1. ^ Schulman J, Wolski F, Dhariwal P, et al. Proximal policy optimization algorithms[J]. arXiv preprint arXiv:1707.06347, 2017.
2. ^ Schulman J, Moritz P, Levine S, et al. High-dimensional continuous control using generalized advantage estimation[J]. arXiv preprint arXiv:1506.02438, 2015.
3. ^ a b c Engstrom L, Ilyas A, Santurkar S, et al. Implementation matters in deep policy gradients: A case study on PPO and TRPO[J]. arXiv preprint arXiv:2005.12729, 2020.
4. ^ Tucker G, Bhupatiraju S, Gu S, et al. The mirage of action-dependent baselines in reinforcement learning[C]//International conference on machine learning. PMLR, 2018: 5015-5024.
5. ^ Chou P W, Maturana D, Scherer S. Improving stochastic policy gradients in continuous control with deep reinforcement learning using the beta distribution[C]//International conference on machine learning. PMLR, 2017: 834-843.

自动滚动

你会PPO算法吗？那这些问题你知道答案吗？

以下部分部分摘录自 你会PPO算法吗？那这些问题你知道答案吗？

PPO算法的全称是什么？

PPO算法的全称是Proximal Policy Optimization，中文可以翻译为近端策略优化。

PPO算法是为了解决什么问题而设计的？

PPO (Proximal Policy Optimization) 算法是为了解决深度强化学习中策略优化的问题而设计的。具体来说，它旨在解决以下几个关键问题：

1. 样本效率：在深度强化学习中，高样本效率是非常重要的。PPO通过使用一种特殊的目标函数，尽量减少所需的样本数量，从而提高样本效率。
2. 稳定性和健壮性：在强化学习中，训练过程往往容易受到不稳定和崩溃的影响。PPO通过限制策略更新的步长，以确保训练过程的稳定性，并避免由于过大的策略更新而导致的性能下降。
3. 简单和通用性：PPO试图在简单和通用性之间找到平衡。它的设计目标是在不同的任务和环境中都能表现良好，且实现简单，容易理解和使用。
4. 解决信任区域优化的问题：传统的信任区域方法（例如TRPO）往往在求解优化问题时需要更多的计算资源和时间。PPO通过引入一种简化的目标函数来避免这些复杂的计算，同时仍然保持了信任区域的思想，即在更新策略时不偏离当前策略太远。

PPO通过其特定的目标函数和优化技术，能够在许多不同的任务和环境中实现高效、稳定的学习，使其成为当前强化学习领域中非常流行和广泛应用的算法之一。

PPO算法相较于其他强化学习算法如DQN或TRPO有什么优势？

PPO (Proximal Policy Optimization) 算法相较于其他强化学习算法如DQN (Deep Q-Network) 或 TRPO (Trust Region Policy Optimization) 有几方面的优势：

1. 稳定性和样本效率：PPO通过限制策略更新的大小，提供了一种稳定的训练过程，避免了大的策略更新可能导致的训练不稳定。相比

之下，DQN可能会在某些情况下表现出不稳定的训练行为。PPO还通过使用旧的策略样本来提高样本效率，而DQN通常需要更多的样本来实现良好的性能。

2. 处理连续动作空间： PPO能够很好地处理连续的动作空间，这使得它在诸如机器人控制和自动驾驶等连续控制任务中特别有用。而DQN更适合于离散动作空间的任务。
3. 简化和实用性：PPO的实现相对简单，容易理解，而且它不需要复杂的调参就能在许多任务中表现良好。**相比之下，DQN和TRPO可能需要更多的调参和技巧来获得良好的性能。**
4. 算法效率：**PPO通过近似解决TRPO中的优化问题，大大减少了计算要求，同时保持了TRPO的核心思想**。这使得PPO在计算效率上具有优势，特别是与TRPO比较。
5. 通用性：PPO被设计为一种通用的算法，能够在多种不同的任务和环境中表现良好。它的目标是在简单性和性能之间找到一个好的平衡，使其成为一种非常实用和广泛应用的算法。
6. 超参数敏感性：PPO通常被认为是相对不太敏感于超参数选择的，这意味着它在不同的任务和环境中可能不需要太多的超参数调整。而DQN和TRPO可能会更敏感于超参数的选择。

通过这些优势，PPO为强化学习社区和实际应用提供了一个高效、稳定和通用的算法选择。

PPO中的clip函数有什么作用?

在PPO（Proximal Policy Optimization，近端策略优化）算法中，clip函数起着至关重要的作用。它是用来确保策略更新保持在一个合理的范围内，以保证训练的稳定性和避免过大的策略变化。以下是clip函数在PPO中的主要作用：

限制策略更新：Clip函数通过限制目标函数的值，从而限制了策略更新的大小。这确保了新策略不会偏离旧策略太远，有助于保持训练过程的稳定性。
避免过优化：在传统的策略梯度方法中，如果单步更新太大，可能会导致过优化和性能下降。Clip函数通过限制单步更新的大小，有助于避免过优化的问题。
增加样本效率：Clip函数通过限制策略的变化，使得旧的样本仍然保持有用。这样，算法可以在多个更新中重复使用同一批样本，从而提高样本效率。
保持稳定和高效的学习：通过控制策略更新的大小，Clip函数帮助保持了学习过程的稳定性和高效性，使得PPO能够在多种不同的任务和环境中取得良好的性能。
具体实现中，PPO的clip函数是通过在目标函数中加入一个clip操作来实现的。该函数将比率（新策略与旧策略的比值）限制在一个预定义的范围内，例如[1-ϵ,1+ϵ]，其中ϵ是一个小的正数（如0.2）。通过这种方式，PPO确保了策略更新的大小在一个合理的范围内，避免了过大的策略变化和可能的不稳定性，从而实现了稳定和高效的学习。

PPO算法中如何保证数据效率?

在PPO（Proximal Policy Optimization）算法中，数据效率是通过多种方式来保证的，以确保在有限的数据样本上实现有效的学习。以下是PPO算法保证数据效率的主要方式：

1. 多次更新: PPO算法通常会对每个数据批次进行多次更新，而不是在每个数据批次上仅执行一次更新。这种方法使得算法能够更好地利用每个数据样本，提高数据效率。
2. 使用旧的策略样本: 在传统的强化学习算法中，通常在每次更新策略后丢弃旧的样本。而PPO利用旧的策略样本来进行多次更新，以此来提高样本的利用率和数据效率。
3. Clip目标函数: 通过引入clip函数，PPO限制了策略更新的大小，确保新策略不会偏离旧策略太远。这种方法允许算法在多个更新中重复使用相同的数据样本，从而提高数据效率。
4. 广义优势估计（GAE）: PPO通常与广义优势估计（GAE）结合使用，以减少估计方差并提高样本效率。GAE能够更准确地估计每个状态和动作的优势，从而提高学习效率。
   稳定的学习过程: 通过保持学习过程的稳定性，PPO能够避免需要额外的数据来修正不稳定的策略更新。稳定的学习过程可以帮助提高数据效率。
5. 效率的优化算法: PPO通过使用效率较高的优化算法，如Adam或TRPO，来确保在有限的数据上实现有效的学习。效率较高的优化算法可以帮助提高数据效率和学习速度。
6. 迷你批次学习: PPO通常使用小批量（mini-batch）学习，它允许算法在收集完一小批数据后立即进行更新，而不是等待整个数据集。这种方法可以加速学习过程，提高数据效率。

通过这些设计和技巧，PPO算法能够在有限的数据上实现有效和稳定的学习，使其成为一个数据效率较高的强化学习算法。

PPO中的值函数和优势函数有什么区别?

简单来说就是优势函数是由rewards函数和值函数计算出来的，参考公式(4)

1.PPO算法概述

PPO算法的核心是使用如下策略损失函数：

$$L^{CLIP}(\theta)=\hat{\mathbb{E}}_t\left[\min\left(r_t(\theta)\hat{A}_t,\operatorname{clip}(r_t(\theta),1-\epsilon,1+\epsilon)\hat{A}_t\right)\right] \tag{1}$$

其中，

$$r_t(\theta)=\frac{\pi_\theta\left(a_t\mid s_t\right)}{\pi_{\theta_{\text{old}}}\left(a_t\mid s_t\right)} \tag{2}$$

表示新旧策略之间的比率。ϵ 为一个超参数，用来确保当利用同一批数据进行多次策略更新时，“新旧策略之间的差距不要太大”。一般情况下，我们设置 $\epsilon=0.2$ 。

(1)式中， $\hat{A}_t$ 为优势函数的估计，根据PPO的原始论文建议，我们一般使用GAE（generalized advantage estimation）[2]来计算优势函数

$$\hat{A}_t^{GAE(\gamma,\lambda)}=\sum_{l=0}^{\infty}(\gamma\lambda)^l\delta_{t+l}^V \tag{3}$$

其中， δ_t^V 为：

$$\delta_t^V=r_t+\gamma V_\omega\left(s_{t+1}\right)-V_\omega\left(s_t\right) \tag{4}$$

PPO算法中的探索策略是如何实现的?

在PPO（Proximal Policy Optimization）算法中，探索是通过随机性来实现的，主要依赖于策略本身的随机性来探索新的状态和动作。探索策略在PPO中的实现主要包括以下几点：

1. **随机策略**: PPO通常使用随机策略，例如softmax策略或高斯策略，以引入探索。随机策略允许PPO在每个状态下尝试多种不同的动作，从而探索环境的不同区域。
2. **熵正则化**: 为了鼓励探索，PPO可以通过在目标函数中添加熵正则项来增加策略的熵。熵正则项鼓励策略保持随机性，从而促进探索。熵正则化的目标函数可以写作：$[L(\theta)=L^{CLIP}(\theta)+\beta H(\pi_\theta)]$ 其中，(H) 是策略的熵，(β) 是熵正则化系数。
3. **值函数和优势函数**: PPO利用值函数和优势函数来评估不同动作的效果，这有助于指导探索策略。通过优势函数，PPO可以识别出那些比平均水平更好或更差的动作，并据此调整探索策略。
4. **学习率调整**: 学习率对探索也有影响。较高的学习率可能会导致更多的探索，而较低的学习率可能会减少探索。在PPO中，可以通过调整学习率来影响探索的程度。
5. **延迟更新和迷你批次**: PPO通常使用延迟更新和迷你批次来实现高效的学习和探索。通过在每个

迷你批次上进行多次更新，PPO可以在探索新动作的同时，有效地利用已收集的数据。

通过以上几种方法，PPO能够实现有效的探索策略，探索环境中不同的状态和动作，从而帮助找到更优的策略。同时，通过合理的探索和利用平衡，PPO能够在各种任务和环境中实现高效和稳定的学习。

PPO如何处理连续动作空间的问题?

在处理连续动作空间的问题时，PPO (Proximal Policy Optimization) 采用一种参数化的策略表示，并通过优化算法来更新策略参数，以此来学习一个高效的策略。以下是PPO处理连续动作空间问题的主要方式:

1. 参数化的策略表示: 在连续动作空间中，PPO通常使用参数化的高斯策略来表示策略。每个动作由一个高斯分布生成，该分布的均值和方差由神经网络参数化，这个神经网络以状态为输入，并输出动作的均值和方差。
2. 学习均值和方差: PPO的目标是学习一个参数化策略的参数，以便在给定状态下生成高效的动作。通过优化算法，PPO能够学习神经网络的参数，从而学习动作的均值和方差，使得生成的动作能够最大化预期的总回报。
3. 稳定的优化: 为了保持训练的稳定性，PPO使用了clip目标函数和熵正则化等技术。clip目标函数通过限制策略更新的大小来保持稳定性，而熵正则化通过增加策略的熵来鼓励探索，这两者都有助于在连续动作空间中实现稳定和高效的学习。
4. 优势函数: 优势函数评估了在给定状态下采取不同动作的相对优势。通过优势函数，PPO能够识别出哪些动作比平均水平更好，从而指导策略的更新和优化。
5. 高效的优化算法: PPO通常使用高效的优化算法，如Adam，来更新策略参数。高效的优化算法能够加速学习过程，并帮助PPO在连续动作空间中找到高效的策略。
6. 重复利用样本: PPO通过重复利用同一批数据进行多次更新，以提高数据效率。这种方法允许PPO在收集新数据之前，充分利用已有数据来更新和优化策略。

通过以上几种方法和技术，PPO能够有效地处理连续动作空间的问题，学习一个能够生成高效动作的参数化策略，从而在各种连续控制任务中实现高效和稳定的学习。

大模型微调实战之 Transformer 强化学习（TRL Reinforcement Learning）（三）Proximal Policy Optimization 大模型与Agent智能体 507
大模型微调实战之 Transformer 强化学习（TRL Reinforcement Learning）（三）Proximal Policy Optimization这是一个基本示例，展示了如何使用库中的...

深入理解大语言模型的微调：trl与peft的结合 步子哥的博客 1058
通过对大语言模型进行微调，我们不仅可以提升模型在特定任务上的表现，还能在有限的计算资源下实现高效的训练。借助trl和peft工具，开发者能够轻松...

trl中的PPO代码解析(炒冷饭版)_trl ppo 8-20
actor loss是基于策略梯度的损失函数,用于优化policy。在ppo中,通常使用一种称为重要性采样(importance sampling)的技术来计算策略梯度。 maximizeθ ...

【慢摹】TRL训练器细节详解(SFT/PPO/DPO/GRPO)_trl训练ppo 7-28
本文总结一下目前TRL典型的训练器的实现细节(SFT,PPO,DPO,GRPO),也是对上一文【速写】PPOTrainer样例与错误思考(少量DAPO)的补充 目前Deep...

TRL(Transformer Reinforcement Learning) PPO Trainer 学习笔记 u012374012的专栏 1013
如上所述，可以使用任何针对字符串返回标量值的函数来生成奖励（the reward can be generated using any function that returns a single value for a strin...

trl中的PPO代码解析（炒冷饭版） 上帝是个娘们的博客 2444